

P0020r1 : Floating Point Atomic

Project: ISO JTC1/SC22/WG21: Programming Language C++
Number: P0020r1
Date: 2016-02-04
Reply-to: hcedwar@sandia.gov
Author: H. Carter Edwards
Contact: hcedwar@sandia.gov
Author: Hans Boehm
Contact: hboehm@google.com
Author: Olivier Giroux
Contact: hboehm@google.com
Author: JF Bastien
Contact: jfb@google.com
Author: James Reus
Contact: reus1@llnl.gov
Audience: SG1 Concurrency
URL: <https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0020.rst>

This paper proposes an extension to the atomic operations library [atomics] for atomic addition on an object conforming to the *atomic-view-concept* (see P0019, Atomic View) instantiated for a floating point type. A class conforming to the *atomic-view-concept* shall also provide the following operations when T is a floating point (FP) type. This capability is critical for high performance computing (HPC) applications.

```
template<> struct atomic< FP > {
    FP fetch_add( FP operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
    FP fetch_add( FP operand , memory_order order = memory_order_seq_cst ) noexcept;
    FP fetch_sub( FP operand , memory_order order = memory_order_seq_cst ) volatile noexcept;
    FP fetch_sub( FP operand , memory_order order = memory_order_seq_cst ) noexcept;
    FP operator+=( FP operand ) volatile noexcept;
    FP operator+=( FP operand ) noexcept;
    FP operator-=( FP operand ) volatile noexcept;
    FP operator-=( FP operand ) noexcept;
};

template<> struct atomic-view-concept < FP > {
    FP fetch_add( FP operand , memory_order order = memory_order_seq_cst ) const noexcept;
    FP fetch_sub( FP operand , memory_order order = memory_order_seq_cst ) const noexcept;
    FP operator+=( FP operand ) const noexcept;
    FP operator-=( FP operand ) const noexcept;
};
```

[Note: The additive operator overloads are consistent with the corresponding overloads specified on `std::atomic<integral>` [29.5, Atomic types]. However, these arithmetic operator overloads are inconsistent with the corresponding operators for corresponding arithmetic type.

```
double d(0)
( d += 1.0 ) += 2.0 ; // d+= returns an lvalue

atomic_view<double> ad(d);
( ad += 1.0 ) += 2.0 ; // error: ad+= returns an rvalue
```

- end note]

FP fetch_add(FP operand , memory_order order = memory_order_seq_cst)
FP fetch_sub(FP operand , memory_order order = memory_order_seq_cst)

Requires: The memory order shall not be `memory_order_acquire`, `memory_order_acq_rel`, or `memory_order_consume`.

Effects: Atomically replaces the value of the floating point object referenced by the atomic view with the sum of the

previous value and operand for *add* or -operand for *sub*. Memory is affected according to the value of **order**. This is an atomic read-modify-write operation (1.10).

Returns: *fetch_op* Atomically, the value of the referenced object immediately before the effects.

Remark: In the event of arithmetic overflow or other floating point exception, the behavior of this operation is implementation defined. It is a quality-of-implementation as to whether arithmetic overflow or other floating point exception is detectable in the resulting value; *e.g.*, a value of NaN.

**** *FP* operator+=(** *FP* operand)**

Effects: Returns **fetch_add(operand) + operand** .

**** *FP* operator-=(** *FP* operand)**

Effects: Returns **fetch_sub(operand) - operand** .